DataFrame Basic Functionality

Let us now understand what DataFrame Basic Functionality is

1. ***T:*** Transposes rows and columns.

2. ***axes:*** Returns a list with the row axis labels and column axis labels as the only members.

3. ***dtypes:*** Returns the dtypes in this object.

4. ***empty:*** True if ND Frame is entirely empty [no items]; if any of the axes are of length 0.

5. ***ndim:*** Number of axes / array dimensions.

6. ***shape:*** Returns a tuple representing the dimensionality of the DataFrame.

7. ***size:*** Number of elements in the NDFrame.

8. ***values:*** Numpy representation of NDFrame.

9. ***head():*** Returns the first n rows.

10. ***tail():*** Returns last n rows.

```
import pandas as pd

import numpy as np

#Create a Dictionary of series
data={'Name':pd.Series(['Tom','James','Ricky','Vin',\
'Steve','Smith','Jack']),'Age':pd.Series([25,26,25,23,\
30,29,23]),'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,\
4.6,3.8])}
```

smetechnologies
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215
FUTURE WITH AI

```
#Create a DataFrame
df = pd.DataFrame(data)
print ("Our data series is:")
print(df)
```

Output:

Our data series is:

	Name	Age	Rating
0	Tom	25	4.23
1	James	26	3.24
2	Ricky	25	3.98
3	Vin	23	2.56
4	Steve	30	3.20
5	Smith	29	4.60
6	Jack	23	3.80

1. T (Transpose)

Returns the transpose of the DataFrame. The rows and columns will interchange.

```
import pandas as pd
import numpy as np
# Create a Dictionary of series
data = {'Name': pd.Series(['Tom', 'James', 'Ricky', 'Vin',\
'Steve', 'Smith', 'Jack']),
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
      'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
      'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20,\
                4.6, 3.8])}
# Create a DataFrame
df = pd.DataFrame(data)
print("The transpose of the data series is:")
print(df.T)
```

Output:

The transpose of the data series is:

	0	1	2	3	4	5	6
Name	*Tom*	*James*	*Ricky*	*Vin*	*Steve*	*Smith*	*Jack*
Age	*25*	*26*	*25*	*23*	*30*	*29*	*23*
Rating	*4.23*	*3.24*	*3.98*	*2.56*	*3.2*	*4.6*	*3.8*

2. axes

Returns the list of row axis labels and column axis labels.

```
import pandas as pd
import numpy as np
#Create a Dictionary of series
data={'Name':pd.Series(['Tom','James','Ricky','Vin',\
     'Steve','Smith','Jack']),
    'Age':pd.Series([25,26,25,23,30,29,23]),\
   'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,4.6,3.8])}
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191

```
#Create a DataFrame
df = pd.DataFrame(data)
print ("Row axis labels and column axis labels are:")
print(df.axes)
```

Output:

Row axis labels and column axis labels are:

[RangeIndex(start=0, stop=7, step=1), Index(['Name', 'Age', 'Rating'], dtype='object')]

3. dtypes

Returns the data type of each column.

```
import pandas as pd
import numpy as np
#Create a Dictionary of series
data={'Name':pd.Series(['Tom','James','Ricky','Vin',\
     'Steve','Smith','Jack']),\
   'Age':pd.Series([25,26,25,23,30,29,23]),\
   'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,4.6,3.8])}
#Create a DataFrame
df = pd.DataFrame(data)
print ("The data types of each column are:")
print(df.dtypes)
```

Output:

The data types of each column are:

Name object

smetechnologies
CODING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 8419 2345

Age int64

Rating float64

dtype: object

4. empty

Returns the Boolean value saying whether the Object is empty or not;

True indicates that the object is empty.

```
import pandas as pd
import numpy as np
# Create a Dictionary of series
d = {'Name': pd.Series(['Tom', 'James', 'Ricky',\
     'Vin', 'Steve', 'Smith', 'Jack']),\
     'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
     'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, \
      3.20, 4.6, 3.8])}
# Create a DataFrame
df = pd.DataFrame(d)
print("Is the object empty?")
print(df.empty)
```

Output:

Is the object empty?

False

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

5. ndim

Returns the number of dimensions of the object. By definition, DataFrame is a 2D object.

```
import pandas as pd
import numpy as np
#Create a Dictionary of series
data = ='Name':pd.Series(['Tom','James','Ricky','Vin',\
          'Steve','Smith','Jack']),\
          'Age':pd.Series([25,26,25,23,30,29,23]),\
         'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,\
          4.6,3.8])}
#Create a DataFrame
df = pd.DataFrame(data)
print ("Our object is:")
print(df)
print ("The dimension of the object is:")
print (df.ndim)
```

Output:

```
Our object is:
    Name  Age  Rating
0    Tom   25    4.23
1  James   26    3.24
2  Ricky   25    3.98
3    Vin   23    2.56
```

smetechnologies
GUIDING THE FUTURE WITH A...
II Floor, Express House, ...loor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
4  Steve   30    3.20

5  Smith   29    4.60

6   Jack   23    3.80

The dimension of the object is:

2
```

6. shape

Returns a tuple representing the dimensionality of the DataFrame. Tuple (a,b), where a represents the number of rows and b represents the number of columns.

```
import pandas as pd
import numpy as np
# Create a Dictionary of series
data = {'Name':pd.Series(['Tom','James','Ricky','Vin',\
'Steve', 'Smith', 'Jack']),\
     'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
     'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20,\
      4.6, 3.8])}
# Create a DataFrame
df = pd.DataFrame(data)
print("Our object is:")
print(df)
print("The shape of the object is:")
print(df.shape)
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191255

Output:

Our object is:

```
    Name  Age  Rating
0    Tom   25    4.23
1  James   26    3.24
2  Ricky   25    3.98
3    Vin   23    2.56
4  Steve   30    3.20
5  Smith   29    4.60
6   Jack   23    3.80
```

The shape of the object is:

(7, 3)

7. size

Returns the number of elements in the DataFrame.

```
import pandas as pd
import numpy as np
# Create a Dictionary of series
data = {'Name':pd.Series(['Tom','James','Ricky','Vin',\
'Steve', 'Smith', 'Jack']),
     'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
     'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20,\
      4.6, 3.8])}
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778601215

```
# Create a DataFrame
df = pd.DataFrame(data)
print("Our object is:")
print(df)
print("The total number of elements in our object is:")
print(df.size)
```

Output:

```
Our object is:
    Name  Age  Rating
0    Tom   25    4.23
1  James   26    3.24
2  Ricky   25    3.98
3    Vin   23    2.56
4  Steve   30    3.20
5  Smith   29    4.60
6   Jack   23    3.80
The total number of elements in our object is:
21
```

8. values

Returns the actual data in the DataFrame as an NDarray.

```
import pandas as pd
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
import numpy as np
# Create a Dictionary of series
data = {'Name': pd.Series(['Tom','James','Ricky','Vin',
'Steve', 'Smith', 'Jack']),\
     'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
     'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, \
      3.20, 4.6, 3.8])}
# Create a DataFrame
df = pd.DataFrame(data)
print("Our object is:")
print(df)
print("The actual data in our data frame is:")
print(df.values)
```

Output:

Our object is:

	Name	Age	Rating
0	Tom	25	4.23
1	James	26	3.24
2	Ricky	25	3.98
3	Vin	23	2.56
4	Steve	30	3.20
5	Smith	29	4.60
6	Jack	23	3.80

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

The actual data in our data frame is:

```
[['Tom' 25 4.23]
 ['James' 26 3.24]
 ['Ricky' 25 3.98]
 ['Vin' 23 2.56]
 ['Steve' 30 3.2]
 ['Smith' 29 4.6]
 ['Jack' 23 3.8]]
```

9. head()

To view a small sample of a DataFrame object, use the head() and tail() methods.Head() returns the first n rows (observe the index values). The default number of elements to display is five, but you may pass a custom number.

```
import pandas as pd
import numpy as np
# Create a Dictionary of series
data={'Name':pd.Series(['Tom','James','Ricky','Vin',\
      'Steve','Smith', 'Jack']),\
     'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
     'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20,\
      4.6, 3.8])}
```

smectechnologies
GUIDING TECH FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7778191215

```
# Create a DataFrame
df = pd.DataFrame(data)
print("Our data frame is:")
print(df)
print("The first two rows of the data frame is:")
print(df.head(2))
```

Output:

```
Our data frame is:

    Name  Age  Rating

0    Tom   25    4.23

1  James   26    3.24

2  Ricky   25    3.98

3    Vin   23    2.56

4  Steve   30    3.20

5  Smith   29    4.60

6   Jack   23    3.80

The first two rows of the data frame is:

    Name  Age  Rating

0    Tom   25    4.23

1  James   26    3.24
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

10. tail()

returns the last n rows (observe the index values). The default number of elements to display is five, but you may pass a custom number.

```
import pandas as pd
import numpy as np
# Create a Dictionary of series
data={'Name':pd.Series(['Tom','James','Ricky','Vin',\
       'Steve', 'Smith', 'Jack']),
      'Age': pd.Series([25, 26, 25, 23, 30, 29, 23]),\
      'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20,\
       4.6, 3.8])}
# Create a DataFrame
df = pd.DataFrame(d)
print("Our data frame is:")
print(df)
print("The last two rows of the data frame is:")
print(df.tail(2))
```

Output:

```
Our data frame is:

    Name  Age  Rating

0    Tom   25    4.23

1  James   26    3.24

2  Ricky   25    3.98
```

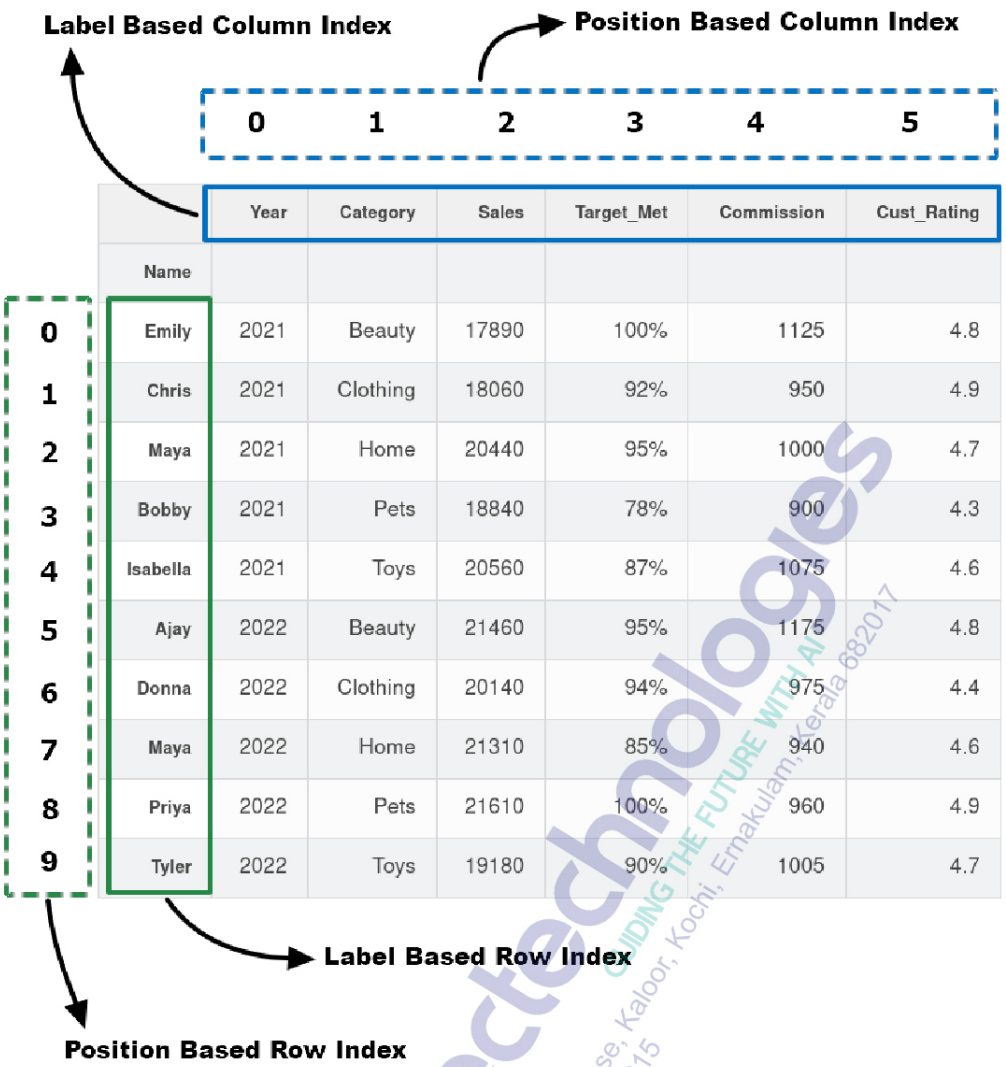
smetechnologies
GUIDING THE FUTURE WITH TECH
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778101215

```
3    Vin   23    2.56
4  Steve   30    3.20
5  Smith   29    4.60
6   Jack   23    3.80
The last two rows of the data frame is:
    Name  Age  Rating
5  Smith   29     4.6
6   Jack   23     3.8
```

Row Selection, Addition, and Deletion

We will now understand row selection, addition and deletion through examples. Let us begin with the concept of selection.

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215



	Name	Year	Category	Sales	Target_Met	Commission	Cust_Rating
0	Emily	2021	Beauty	17890	100%	1125	4.8
1	Chris	2021	Clothing	18060	92%	950	4.9
2	Maya	2021	Home	20440	95%	1000	4.7
3	Bobby	2021	Pets	18840	78%	900	4.3
4	Isabella	2021	Toys	20560	87%	1075	4.6
5	Ajay	2022	Beauty	21460	95%	1175	4.8
6	Donna	2022	Clothing	20140	94%	975	4.4
7	Maya	2022	Home	21310	85%	940	4.6
8	Priya	2022	Pets	21610	100%	960	4.9
9	Tyler	2022	Toys	19180	90%	1005	4.7

smectechnologies
GUIDING THE FUTURE WITH AI
Kaloor, Kochi, Ernakulam, Kerala 682017
+91 9778191215

Selection by Label

Label-based indexing: This means that you use the labels (names) of rows and columns to select data.

Syntax:

df.loc[row_label, column_label] or df.loc[start_label:end_label, start_col:end_col].

```
import pandas as pd
data = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),\
        'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c',\
        'd'])}
df = pd.DataFrame(data)
print(df.loc['b'])
```

Output:

```
   one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4


one    2.0
two    2.0
Name: b, dtype: float64
```

Selection by integer location

Rows can be selected by passing integer location to an iloc function.

```
import pandas as pd
data = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
   'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
df = pd.DataFrame(d)
print(df)
print(df.iloc[2])
```

Output:

```
   one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4
one    3.0
two    3.0
Name: c, dtype: float64
```

Slice Rows

Multiple rows can be selected using ‘ :’ operator.

```
import pandas as pd
d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
   'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}
df = pd.DataFrame(d)
print(df)
print(df[2:4])
```

output:

```
one  two
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191210

```
a  1.0    1
b  2.0    2
c  3.0    3
d  NaN    4
   one   two
c  3.0    3
d  NaN    4
```

Example:

```
import pandas as pd
from sklearn.datasets import load_iris
# Load the Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
# Add the target (species) column to the DataFrame
df['species'] = iris.target
# Sample output of the first few rows
print("First few rows of the Iris dataset:")
print(df.head())
```

Output:

```
First few rows of the Iris dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)   Species
```

smrctechnologies
GUIDING THE FUTURE WITH AI
Kaloor, Kochi, Ernakulam, Kerala 682017

0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
# Using loc (label-based indexing)
print("\nUsing loc:")
# Select the species of the 3rd row (index 2)
print(df.loc[2, 'species'])
# Select rows from 1 to 3 and columns 'sepal length (cm)' and
'species'
print(df.loc[1:3, ['sepal length (cm)', 'species']])
# Using iloc (position-based indexing)
print("\nUsing iloc:")
# Select the value at the 3rd row and 2nd column (position
2,1) – 'petal width (cm)' value
print(df.iloc[2, 1])
# Select rows from position 0 to 3 (exclusive of 4) and the
first two columns (position 0 and 1)
print(df.iloc[0:4, [0, 1]])
```

Output:

Using loc:

0

sepal length (cm) species

smctechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97781 11215

```
1                 4.9         0

2                 4.7         0

3                 4.6         0
```

```
Using iloc:

3.2

   sepal length (cm)  sepal width (cm)

0                 5.1                3.5

1                 4.9                3.0

2                 4.7                3.2

3                 4.6                3.1
```

.loc() Selection by location

Pandas provide various methods to have purely label based indexing. When slicing, the start bound is also included. Integers are valid labels, but they refer to the label and not the position.

.loc() has multiple access methods like –

A single scalar label

A list of labels

A slice object

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7736201215

A Boolean array

loc takes two single/list/range operator separated by ','. The first one indicates the row and the second one indicates columns.

Select all rows for a specific column

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4), \
     index = ['a','b','c','d','e','f','g','h'], \
     columns = ['A', 'B', 'C', 'D'])
print("Original Df")
print(df)
print(df.loc[:,'A'])
```

Output:

Original Df

```
          A         B         C         D
a  1.020071 -0.562052  0.183077  0.141951
b -0.269729  0.153034 -1.360383  0.990632
c  1.301864 -0.108967  0.000808 -2.038066
d  0.984603 -1.517982  0.573402 -0.834571
e  0.102339 -0.154979 -0.085712 -0.220676
f -0.551101  0.452989  0.803740  0.441640
g -1.707507 -0.965319 -1.131366  0.247122
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 7907191215

```
h -1.204713 -0.518586 -0.402809 -0.390756
```

```
a    1.020071
b   -0.269729
c    1.301864
d    0.984603
e    0.102339
f   -0.551101
g   -1.707507
h   -1.204713
Name: A, dtype: float64
```

Select all rows for multiple columns

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4), \
     index = ['a','b','c','d','e','f','g','h'], \
     columns = ['A', 'B', 'C', 'D'])
print(df.loc[:,['A','C']])
```

Output:

```
          A         C
a -0.411563  1.588357
b  0.698673  0.679170
c -0.122504  0.807433
```

smctechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778891212

```
d  0.588982  1.237754
e -1.354231 -0.481274
f  0.384187 -0.645318
g  0.126425  0.178756
h -0.150078  0.162449
```

Select few rows for multiple columns

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),\
     index = ['a','b','c','d','e','f','g','h'], \
     columns = ['A', 'B', 'C', 'D'])
print(df.loc[['a','b','f','h'],['A','C']])
```

Output:

```
A         C
a -0.007973  0.428263
b -0.395085 -0.060454
f -0.670361 -0.260289
h  0.262532  0.982931
```

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
# import the pandas library and aliasing as pd
```

Select range of rows for all columns

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),\
     index = ['a','b','c','d','e','f','g','h'], \
     columns = ['A', 'B', 'C', 'D'])
print(df.loc['a':'h'])
```

Output:

```
          A         B         C         D
a -0.779965  2.048826  0.362352 -0.350989
b -0.061612 -0.524133 -1.894138 -0.996092
c  0.110118  0.042171  0.228839  0.415355
d  1.217177 -0.362562 -0.188985 -0.290914
e -0.985426  0.704533 -0.022084  0.651486
f  1.176105 -2.035761 -0.806778 -0.030166
g -0.703882 -1.461334 -0.802974  0.895749
h -0.729383  0.775261  0.200570 -0.114541
```

Getting values with a boolean array

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),
```

```
index = ['a','b','c','d','e','f','g','h'], columns = ['A',
'B', 'C', 'D'])
print(df.loc['a']>0)
```

Output:

```
A     True
B     True
C     True
D    False
Name: a, dtype: bool
```

.iloc() Selection by Integer location

Pandas provide various methods in order to get purely integer based indexing. Like python and numpy.

The various access methods are as follows :

An Integer

A list of integers

A range of values

A list of integers

A range of values

smectechnologies
GOING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

Select 4 rows for all columns

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4), \
     columns = ['A', 'B', 'C', 'D'])
print(df.iloc[:4])
```

Output:

```
          A         B         C         D
0 -0.493503  0.296914  0.922541  0.414268
1  0.296769  0.847003  0.052303 -0.200299
2 -0.592017  0.600889 -0.691062 -0.614279
3  0.798952 -1.019599  0.655656 -1.402694
```

Selecting a range of rows and columns

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4), columns = ['A', 'B',
'C', 'D'])
# Integer slicing
print(df.iloc[:4])
print(df.iloc[1:5, 2:4])
```

Output:

```
  A         B         C         D
0 -0.808761  0.457610 -1.874859  0.346348
```

smetechnologies
GUIDING THE FUTURE WITH IT
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778102215

```
1 -1.113190  1.042210 -0.497639  0.670518
2 -0.853154  0.389452  0.186796  1.009133
3  0.683928 -1.232726  0.788916  1.356914
          C         D
1 -0.497639  0.670518
2  0.186796  1.009133
3  0.788916  1.356914
4 -0.122763 -0.062472
```

Slicing through list of values

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4), columns = ['A', 'B',
'C', 'D'])
print(df.iloc[[1, 3, 5], [1, 3]])
print(df.iloc[1:3, :])
print(df.iloc[:,1:3])
```

Output:

```
 B         D
1 -1.583803  0.635706
3  0.012548 -0.773214
5  0.817090  0.768234
          A         B         C         D
```

snetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
1 -1.199506 -1.583803 -0.500844  0.635706

2 -0.432852  1.218018 -1.447651 -1.131431

          B         C

0  1.175049  1.137804

1 -1.583803 -0.500844

2  1.218018 -1.447651

3  0.012548 -1.660108

4 -1.171807 -0.249070

5  0.817090  0.021319

6  0.326512  2.064234

7 -0.679426 -2.698323
```

Deletion of Rows

Use index label to delete or drop rows from a DataFrame. If label is duplicated, then multiple rows will be dropped.

```
import pandas as pd
df = pd.DataFrame([[1, 2], [3, 4]], columns = ['a','b'])
print(df)
Output:
    a  b
0  1  2
1  3  4
```

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express Pulse, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778107215

```
# Drop rows with label 0
df = df.drop(0)
print(df)
```

Output:

df

```
    a   b
1   3   4
```

Pandas - Descriptive Statistics

Functions & Description

1. Measures of central tendency: These describe the center or typical value of a dataset, such as:

- **Mean**: The average of the data.
- **Median**: The middle value when the data is ordered.
- **Mode**: The most frequently occurring value.

2. Measures of spread or variability: These describe the dispersion or spread of the data, such as:

- **Variance**: The average squared deviation from the mean.
- **Standard deviation**: The square root of variance, indicating how spread out the values are.
- **Range**: The difference between the maximum and minimum values.

3. Shape of the distribution: These measures describe the shape of the distribution of the data, such as:

smeetechnologies
GUIDING THE FUTURE WITH AI
Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
No: +91 97

- **Skewness**: Measures the asymmetry of the distribution (whether the data is skewed to the left or right).
- **Kurtosis**: Measures the "tailedness" of the distribution.

4. Quantiles: These are values that divide the data into different parts. Common quantiles include:

- **Quartiles** (Q1, Q2, Q3): Divides the data into four equal parts.
- **Percentiles**: Divides the data into 100 equal parts.

Let us now understand the functions under Descriptive Statistics in Python Pandas:

Sl.No.	Function	Description
1	`count()`	Number of non-null observations
2	`sum()`	Sum of values
3	`mean()`	Mean of Values
4	`median()`	Median of Values
5	`mode()`	Mode of values
6	`std()`	Standard Deviation of the Values
7	`min()`	Minimum Value
8	`max()`	Maximum Value
9	`abs()`	Absolute Value
10	`prod()`	Product of Values

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191216

11	*cumsum()*	Cumulative Sum
12	*cumproduct()*	Cumulative product
13	*cov()*	Covariance
14	*corr()*	Correlation
15	*skew()*	Skewness
16	*kurt()*	Kurtosis

```
import pandas as pd
import numpy as np
from scipy.stats import skew, kurtosis

def descriptive_statistics(df, column):
    # Calculate various statistics
    mean = df[column].mean()
    median = df[column].median()
    mode = df[column].mode()[0]  # Mode returns a series, get
the first element
    variance = df[column].var()
    std_dev = df[column].std()
    data_range = df[column].max() - df[column].min()
    min_value = df[column].min()
    max_value = df[column].max()
    skewness = skew(df[column])
```

smartechnologies
BUILDING THE FUTURE WITH AI
Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
No: +91 6782 9215

```
        kurt = kurtosis(df[column], fisher=True)  # Excess
#kurtosis (normal distribution = 0)
    # Calculate percentiles (25th, 50th, 75th)
    percentiles = [ \
        df[column].quantile(0.25),\
        df[column].quantile(0.50),\
        df[column].quantile(0.75)\
    ]

    # Prepare a dictionary to return the results
    statistics = {\
        'Mean': mean,\
        'Median': median,\
        'Mode': mode,\
        'Variance': variance,\
        'Standard Deviation': std_dev,\
        'Range': data_range,\
        'Min': min_value,\
        'Max': max_value,\
        'Skewness': skewness,\
        'Kurtosis': kurt,\
        '25th Percentile': percentiles[0],\
        '50th Percentile (Median)': percentiles[1],\
        '75th Percentile': percentiles[2]\
    }
```

smartechnologies
GUIDING THE FUTURE WITH AI
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

```
 # Return the statistics as a pandas Series for better readability
    return pd.Series(statistics)
# Create a Dictionary of series
d = {'Name': pd.Series(['Tom', 'James', 'Ricky',\
            'Vin', 'Steve', 'Smith', 'Jack','Lee', 'David', \
            'Gasper', 'Betina', 'Andres']),
     'Age': pd.Series([25, 26, 25, 23, 30, 29, 23, 34,\
           40, 30, 51, 46]),
     'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20, 4.6,\
              3.8, 3.78, 2.98, 4.80, 4.10, 3.65])}
# Create a DataFrame
df = pd.DataFrame(d)

# Get descriptive statistics for the 'Age' column
age_statistics = descriptive_statistics(df, 'Age')

# Display the result
print(age_statistics)
```

Output:

Mean	*31.833333*
Median	*29.500000*
Mode	*23.000000*
Variance	*85.242424*
Standard Deviation	*9.232682*

smetechnologies
GUIDING THE FUTURE WITH
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 68201
Ph. No: +91 9778191215

```
Range                          28.000000
Min                            23.000000
Max                            51.000000
Skewness                        0.987968
Kurtosis                       -0.304630
25th Percentile                25.000000
50th Percentile (Median)       29.500000
75th Percentile                35.500000
dtype: float64
```

25th Percentile: This means 25% of the data is below 25

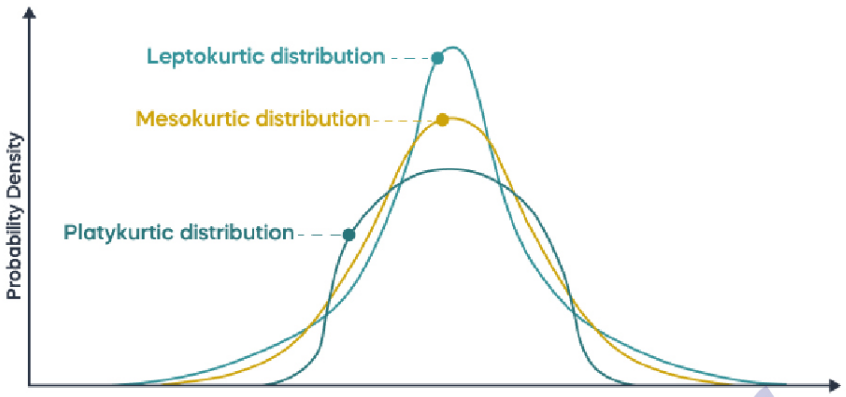
50th Percentile: This means 50% of the data is below 29.5, and it's the middle point of the dataset

75th Percentile: This means 75% of the data is below 35.5, and 25% is above this value

Skewness : As the value increases (becomes more positive), the right tail of the distribution becomes longer or more spread and vice versa. If It is zero then its symmetric.

Kurtosis : When kurtosis is greater than 0, the distribution is called leptokurtic. A positive kurtosis indicates that the distribution has a higher peak than a normal distribution, with fatter tails. This means that there are more outliers or extreme values far from the mean.

smetechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kochi, Ernakulam, Kerala 682017
Ph. No.+91 9778191215



A kurtosis of 0 indicates a normal distribution (the classic bell curve). This means the distribution has a moderate peak and normal tails—neither too heavy nor too light. The tails are of normal weight, and the data is spread out in a predictable way. When kurtosis is negative, the distribution is called platykurtic. A negative kurtosis indicates that the distribution is flatter than a normal distribution, with lighter tails. This means there are fewer extreme values or outliers compared to a normal distribution.

cumsum() and cumproduct()

```
import pandas as pd
# Sample DataFrame with some values
data = {'Value': [2, 3, 5, 7, 11, 13]}
df = pd.DataFrame(data)

# Calculate Cumulative Sum
df['Cumulative Sum'] = df['Value'].cumsum()

# Calculate Cumulative Product
df['Cumulative Product'] = df['Value'].cumprod()
```

```
# Display the DataFrame with Cumulative Sum and Cumulative
Product
print("DataFrame with Cumulative Sum and Cumulative \
Product:")
print(df)
```

Output:

DataFrame with Cumulative Sum and Cumulative Product:

	Value	Cumulative Sum	Cumulative Product
0	2	2	2
1	3	5	6
2	5	10	30
3	7	17	210
4	11	28	2310
5	13	41	30030

smectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215

cov() and corr()

```
import pandas as pd
# Creating the DataFrame
d = {'Name': pd.Series(['Tom', 'James', 'Ricky', 'Vin',\
          'Steve', 'Smith', 'Jack', 'Lee', 'David', \
          'Gasper','Betina', 'Andres']),\
```

```
    'Age': pd.Series([25, 26, 25, 23, 30, 29, 23, 34, 40, \
           30, 51, 46]),\
    'Rating': pd.Series([4.23, 3.24, 3.98, 2.56, 3.20, \
           4.6, 3.8, 3.78, 2.98, 4.80, 4.10, 3.65])\
}
df = pd.DataFrame(d)
# Calculating the correlation and covariance
correlation = df[['Age', 'Rating']].corr()
#print(df['Age'].corr(df['Rating']))
covariance = df[['Age', 'Rating']].cov()
print("Correlation:\n", correlation)
print("\nCovariance:\n", covariance)
```

output:

Correlation:

	Age	*Rating*
Age	*1.000000*	*0.058586*
Rating	*0.058586*	*1.000000*

Covariance:

	Age	*Rating*
Age	*85.242424*	*0.357879*
Rating	*0.357879*	*0.437*

smetechnologies
GUIDING THE FUTURE WITH AI
1st Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph No: +91 9778191215

The covariance between Age and Rating is 0.357879. Since this value is positive, it indicates that Age and Rating tend to increase together. This means that as Age increases, Rating also tends to increase. However, the relationship is not very strong, since covariance values are not standardized and don't provide a direct indication of the strength of the relationship without further context. Magnitude: The covariance is relatively small, suggesting a weak to moderate positive relationship between Age and Rating, though it's not as strong as a higher covariance would be. It's an indication that the two variables have a positive linear relationship, but not necessarily a very strong one

Covariance of 'Age' with itself (85.242424). This is the variance of the 'Age' column. A variance of 85.242424 means that the Age values are spread out quite a bit from the mean. The higher the variance, the more variability there is in the data. The variance of Age is higher compared to the covariance with Rating, meaning that Age has more spread (more variability) than Rating

Covariance of 'Rating' with itself (0.437752).This is the variance of the 'Rating' column. A variance of 0.437752 means that Rating is less spread out than Age. The smaller value suggests that the data points in Rating are more clustered around the mean compared to Age.

Correlation measures the relationship between two or more variables. In statistics, correlation refers to the strength and direction of a linear relationship between variables. Correlation values range from -1 to +1, where:

+1 indicates a perfect positive linear relationship (as one variable increases, the other increases proportionally).-1 indicates a perfect negative linear relationship (as one variable increases, the other decreases proportionally).

0 indicates no linear relationship between the variables.

The correlation between the columns 'Age' and 'Rating' is 0.387422. This indicates a moderate positive correlation between Age and Rating. As Age increases, Rating tends to increase, but the relationship is not very strong (it's below 0.5, so it's not a very high correlation).

- 0 to 0.3: Weak positive correlation (indicating a slight increase in Rating as Age increases).
- 0.3 to 0.7**:** Moderate positive correlation (like your value, suggesting a noticeable trend, but not a very strong relationship).
- 0.7 to 1: Strong positive correlation.
- -1 to -0.7: Strong negative correlation.
- -0.7 to -0.3: Moderate negative correlation.
- -0.3 to 0: Weak negative correlation.

Summarizing Data

The describe() function computes a summary of statistics pertaining to the DataFrame columns.

```
import pandas as pd
import numpy as np
#Create a Dictionary of series
```

```
d =
{'Name':pd.Series(['Tom','James','Ricky','Vin','Steve','Smit
h','Jack',
   'Lee','David','Gasper','Betina','Andres']),
   'Age':pd.Series([25,26,25,23,30,29,23,34,40,30,51,46]),
   'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,4.6,3.8,3.78
,2.98,4.80,4.10,3.65])
}

#Create a DataFrame
df = pd.DataFrame(d)
print (df.describe())
```

Output:

```
       Age      Rating
count  12.000000  12.000000
mean   31.833333   3.743333
std     9.232682   0.661628
min    23.000000   2.560000
25%    25.000000   3.230000
50%    29.500000   3.790000
75%    35.500000   4.132500
max    51.000000   4.800000
```

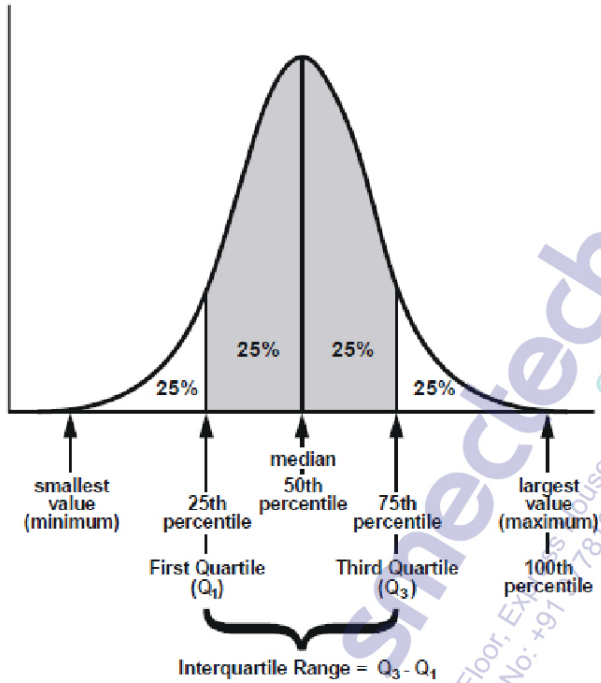
smetechnologies
GUIDING THE FUTURE WITH AI
Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
No: +91 9778191215

The describe() function gives the mean, std and IQR values. And, function excludes the character columns and given summary about numeric columns. 'include' is the argument which is used to pass necessary information regarding what columns need to be considered for summarizing. Takes the list of values; by default, 'number'.

object − Summarizes String columns

number − Summarizes Numeric columns

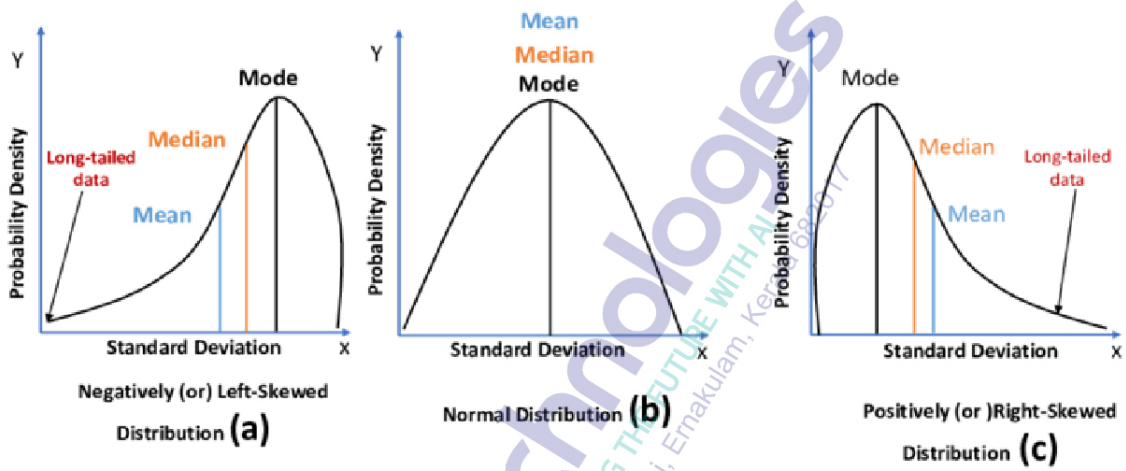
all − Summarizes all columns together (Should not pass it as a list value)



1. **Find the First Quartile (Q1)**: This is the value below which 25% of the data fall.
2. **Find the Third Quartile (Q3)**: This is the value below which 75% of the data fall.
3. **Calculate the Interquartile Range (IQR)**: IQR=Q3−Q1

snectechnologies
GUIDING THE FUTURE WITH AI
II Floor, Exp... House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 ... 91215

- The quartile deviation gives you an idea of the variability of the middle 50% of the data.
- A smaller quartile deviation indicates that the data points are closer to the median, while a larger quartile deviation suggests more spread out data.



IQR Advantages

- It is less affected by outliers compared to other measures of dispersion like the standard deviation, making it a robust measure for skewed distributions

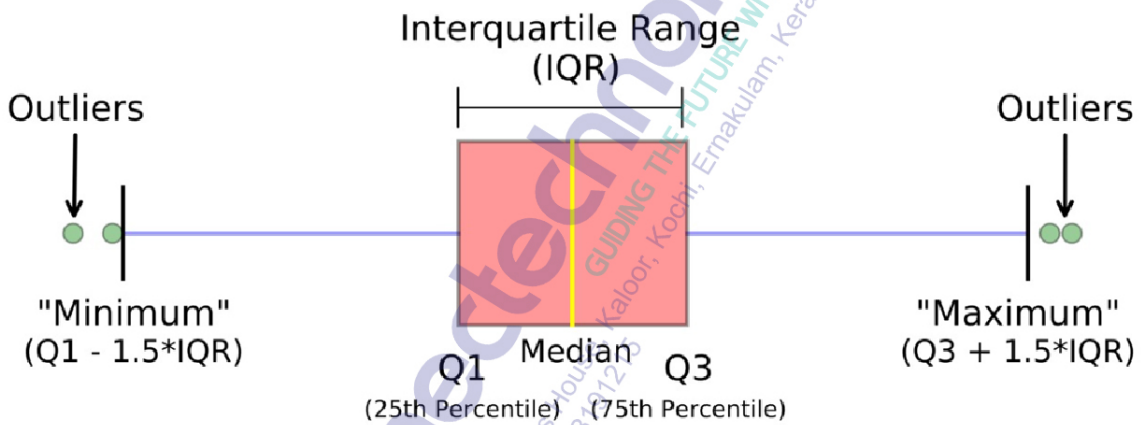
Removal of Outliers

Outliers are data points that are significantly different from the rest of the data in a dataset. These values are either much higher or much lower than most of the other data points, and they can distort statistical analyses, leading to misleading conclusions.

smctechnologies
GUIDING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 97 4191 4115

1. Visual Inspection (Boxplots and Scatterplots)

Boxplot: A boxplot is a simple way to visualize the distribution of data and highlight potential outliers (values outside of the whiskers). While not strictly a programmatic approach, visualizing data can help identify outliers that may not be captured by statistical methods alone. In a boxplot, the central box represents the interquartile range (IQR), which includes the values between the first quartile (Q1) and the third quartile (Q3). Any data points outside of this range are considered outliers. In your data, the value 1000 is much higher than the other values and would likely fall outside the upper whisker, which is why it will be marked as an outlier.



2. Remove Outliers Using IQR (Interquartile Range)

The **Interquartile Range (IQR)** is a measure of statistical dispersion, which describes the range within which the middle 50% of the values lie. It is based on the **quartiles** of the data.

- **Q1 (First Quartile)**: The median of the lower half of the data. It represents the 25th percentile (i.e., 25% of the data points are below this value).
- **Q3 (Third Quartile)**: The median of the upper half of the data. It represents the 75th percentile (i.e., 75% of the data points are below this value).
- **IQR**: The difference between the third quartile (Q3) and the first quartile (Q1). It is calculated as:

$$IQR = Q3 - Q1$$

The IQR can help detect outliers by defining the range within which most of the data lies. Outliers are typically defined as values that fall below a lower threshold or above an upper threshold.

Outlier Detection Formula:

Lower Bound (for detecting outliers below):

$$\text{Lower Bound} = Q1 - 1.5 * IQR$$

Upper Bound (for detecting outliers above):

$$\text{Upper Bound} = Q3 + 1.5 * IQR$$

Any data points below the **Lower Bound** or above the **Upper Bound** are considered outliers.

smectechnologies
GUIDING THE FUTURE WITH I
Kochi, Ernakulam, Kerala 682017
Ph No: +91 7778191215

The multiplier **1.5** is a commonly used value to detect outliers. It is considered a balance between being too strict (removing too many points) and too lenient (leaving too many points).

```
import pandas as pd
# Example DataFrame
data = {'column1': [10, 20, 30, 40, 100, 200, 300, 500, 1000]}
df = pd.DataFrame(data)

# Calculate Q1 (25th percentile), Q3 (75th percentile), and
IQR
Q1 = df['column1'].quantile(0.25)
Q3 = df['column1'].quantile(0.75)
IQR = Q3 - Q1

# Define lower and upper bounds to identify outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Filter out outliers
df_no_outliers = df[(df['column1'] >= lower_bound) & \
(df['column1'] <= upper_bound)]
print(df_no_outliers)
```

3. Remove Outliers Using modified Z-Score

Another method is to use the Z-score, which measures how many standard deviations a data point is from the mean. Commonly, a Z-score above 3 or

smetechnologies
GUIDING THE FUTURE WITH AI
Kaloor, Kochi, Ernakulam, Kerala 682017
Ph.No: +91 9778191215

below -3 is considered an outlier. Good especially for non-normal distributions.

```
import numpy as np
# Example DataFrame
data = {'column1': [10, 20, 30, 40, 100, 200, 300, 500, 1000]}
df = pd.DataFrame(data)

# Calculate the median and MAD
median = np.median(df['column1'])
mad = np.median(np.abs(df['column1'] - median))

# Calculate the modified Z-score
modified_z_scores = 0.6745 * (df['column1'] - median) / mad

# Filter out rows with modified Z-scores greater than 3.5
#(absolute value)
df_no_outliers = df[abs(modified_z_scores) <= 3.5]
print(df_no_outliers)
```

smctechnologies
GIVING THE FUTURE WITH AI
II Floor, Express House, Kaloor, Kochi, Ernakulam, Kerala 682017
Ph. No: +91 9778191215